

MODULE 7 — File Systems

Senior SWE Interview Track — Operating Systems

7.1 File System Basics

1. Why Interviewers Ask This

Backend engineers live on top of file systems (DB storage engines, logs, object stores). Interviewers test whether you know what actually happens on `open/read/write` and where durability really comes from.

2. Core Concept

A file system maps names → files → disk blocks, layered as: **VFS** (uniform API over ext4/XFS/NFS/tmpfs) → concrete FS (allocation, layout, journaling) → block layer (I/O scheduling) → device. Core objects: **superblock** (FS metadata), **inode** (per-file metadata), **dentry** (name → inode), **data blocks**.

3. Internal Working

`open("/var/log/app.log")`: VFS walks the path (dentry cache first), permission-checks each component, loads the inode, creates a `struct file` (offset + flags), installs it in the process FD table, returns the index (the fd). `read()` goes fd → file → inode → block mapping (extents) → **page cache** (hit: memcpy; miss: block I/O then memcpy). `write()` typically dirties page-cache pages; actual disk writes happen later (writeback) unless you `fsync` — the root of all durability questions.

4. ASCII Diagram

```

open/read/write/fsync (syscalls)
  v
[ VFS ] -- dentry cache / inode cache
  v
[ext4] [xfs] [nfs] [tmpfs] [procfs]
  v
[ page cache ] <- most reads/writes stop here (RAM!)
  v (writeback / readahead)
[ block layer: scheduler, merging ]
  v
[ NVMe/SSD/HDD, on-device write cache ]

```

5. Real Production Example

Kafka's speed secret: sequential appends + page cache + `sendfile` zero-copy — "disk" throughput at RAM speed. Postgres/MySQL fight the opposite battle: forcing durability (`fsync/O_DIRECT`) despite the cache.

6. Advantages

Uniform API over wildly different backends; page cache makes hot I/O RAM-fast; names/permissions/hierarchy for free.

7. Trade-offs

The cache lies about durability (`write()` ≠ on disk); many layers to reason about during incidents; POSIX semantics (atomicity of rename, not of write) are subtle.

8. Common Mistakes

- Believing a returned `write()` is durable (it's in RAM; power loss eats it).
- Not knowing `rename()` is atomic (the foundation of safe config/file swaps) while multi-block `write()` is not.
- Forgetting directory `fsync`: creating a file durably requires `fsync` of the *directory* too.

9. Performance Implications

Page-cache hit $\approx$ memcpy (GB/s); miss $\approx$ device latency (NVMe $\sim 100\ \mu\text{s}$, HDD $\sim 10\ \text{ms}$). Free RAM is not wasted — it's cache (`free -h` “available” vs “free” — classic misreading).

10–11. Interview & Follow-ups

- “What happens on `open()` end-to-end?” “When is my write actually on disk?” “Why does Kafka read at memory speed?”
- Follow-ups: “What does the dentry cache buy?” “Atomic file replace pattern?” (write temp $\rightarrow$ fsync $\rightarrow$ rename $\rightarrow$ fsync dir)

12. Coding/Debugging Scenario

Config corruption after power loss $\rightarrow$ app wrote in place without fsync/rename; fix with the atomic-replace pattern.

13. Best Practices

Durability = fsync (file **and** directory); atomic updates via rename; treat page cache as your friend for throughput and your enemy for durability reasoning.

14. Practice Questions

1. Write pseudocode for crash-safe “replace config file”.
2. Trace `cat /etc/hosts` through every layer named above.

7.2 Inode

1. Why Interviewers Ask This

“What’s an inode?” is a Linux screening staple; the deeper hooks — hard links, deleted-but-open files, inode exhaustion — are real incident material.

2. Core Concept

The inode is a file’s identity and metadata record: type, permissions, owner, size, timestamps, **link count**, and pointers to data blocks. It contains **no filename** — names live in directory entries (name $\rightarrow$ inode number). Multiple names can reference one inode = **hard links**.

3. Internal Working

- Classic layout: 12 direct block pointers + single/double/triple indirect (ext2/3); ext4 uses **extents** instead (start + length runs).
- Directory = a file whose contents are (name, inode#) pairs.
- Deletion: `unlink()` decrements link count; data blocks are freed only when link count = 0 **and** no process holds it open — hence “deleted” log files still consuming disk (visible via `ls -l +L`; `du` vs `df` disagree).
- Inodes are a fixed pool created at mkfs (ext4) — you can run out of inodes with disk space free (millions of tiny files).

4. ASCII Diagram

```

dir entry: ("app.log", #5231)      dir entry: ("app_hardlink", #5231)
      \                             /
       v                             v
      inode #5231 [mode,uid,size,mtime,links=2]
        | extents / block ptrs
        v
      [data blocks...]
unlink both + still open by PID 42 -> blocks freed only on close
Symlink: its OWN inode whose data = "/path/to/target" (can dangle)

```

5. Real Production Example

- “Disk full but `du` shows 40% used” → deleted-but-open file held by a running process (restart/`truncate` via `/proc/<pid>/fd/N`).
- “No space left on device” with free GBs → inode exhaustion (`df -i`) from millions of small files (session files, mail queues).
- Log rotation (`logrotate`) exists precisely because of unlink-vs-open semantics (`copytruncate` vs `move+signal`).

6. Advantages

Name/identity separation enables hard links, atomic rename, open-file stability across renames; fixed-size record enables fast stat.

7. Trade-offs

Fixed inode pool (ext4); hard links complicate reasoning (“which name is real?” — none is); metadata-heavy workloads (small files) bottleneck on inode/dentry ops.

8. Common Mistakes

- “The inode stores the filename.”
- Hard link vs symlink confusion: hard link = same inode, same FS only, survives target deletion; symlink = separate inode holding a path, can cross FS and dangle.
- Not knowing `df` vs `du` divergence causes.

9. Performance Implications

`stat`-heavy workloads (web servers checking mtimes, build tools) live on the inode/dentry caches; a `find /` can evict them. Directories with millions of entries degrade lookup (ext4 htree helps; still, shard directories).

10–11. Interview & Follow-ups

- “Hard vs soft link — table.” “What happens on `rm` of an open file?” “Why can a disk be ‘full’ with space free?”

12. Coding/Debugging Scenario

Disk filling with nothing visible: `ls -l | sort -k7 -n` → 80 GB deleted log held by java → rotate properly (reopen signal) or truncate through `/proc/<pid>/fd`.

13. Best Practices

Rotate logs with reopen (SIGHUP/copytruncate); shard huge directories; monitor `df -i`; use `ls -li/stat` fluently.

14. Practice Questions

1. Draw the state after: `touch a; ln a b; ln -s a c; rm a` — what does each of b, c resolve to?
2. mkfs choices for a filesystem storing 500M 2 KB files — what do you tune? (inode ratio, or use a different design entirely)

7.3 Journaling

1. Why Interviewers Ask This

Crash consistency is a systems-design fundamental — the same write-ahead-log idea powers databases, and interviewers love the “what happens on power loss mid-write?” scenario.

2. Core Concept

A multi-block FS update (e.g., append: data block + inode size + bitmap) is not atomic — a crash mid-way corrupts. **Journaling** = write-ahead logging: record the intent in a sequential journal, commit it, *then* apply in place. Recovery replays committed transactions and discards partial ones — mount-time recovery in seconds vs hours of full-disk `fsck`.

3. Internal Working

1. Write transaction (affected metadata ± data) to the journal (sequential).
2. Write **commit record** — the transaction is now atomic-durable.
3. **Checkpoint**: apply blocks to home locations.
4. Reclaim journal space. Crash before commit → transaction ignored; after commit → replayed idempotently. Modes (ext4): **journal** (data+metadata journaled — safest, double-write cost), **ordered** (default: only metadata journaled, but data written *before* the metadata commits — no garbage-in-file), **writeback** (metadata only, data anytime — fastest, may expose stale data post-crash).

4. ASCII Diagram

```
append to file = 3 in-place writes: [data][inode][bitmap] (not atomic!)
Journaling:
  JOURNAL (sequential): | T42: data,inode,bitmap | COMMIT T42 |
  crash here? --^ no commit -> ignore (file unchanged, consistent)
  after commit -> replay T42 -> in-place writes -> checkpoint done
Same idea as: DB WAL / redo log / Raft log.
```

5. Real Production Example

ext4/XFS journals (XFS: metadata-only by design); databases replicate the pattern (Postgres WAL, InnoDB redo log) — and *stack* on it: DB WAL on a journaled FS (why DBs often use O_DIRECT to avoid double-buffering, and fdatasync to avoid double journaling costs). Copy-on-write filesystems (ZFS, Btrfs) achieve crash consistency differently — never overwrite in place.

6. Advantages

Seconds-fast recovery; metadata always consistent; sequential journal writes are cheap; atomicity building block.

7. Trade-offs

Write amplification (up to 2× in full-journal mode); journal is a serialization point; ordered mode still doesn't make *your* data transactional — only FS structures (your app still needs fsync + its own WAL for app-level atomicity).

8. Common Mistakes

- Believing journaling makes application writes atomic/durable (it protects FS *metadata* consistency; your data needs fsync and app-level protocols).
- Not knowing the three ext4 modes and the default (ordered).
- Confusing journaling (redo intent log) with CoW snapshots (ZFS/Btrfs).

9. Performance Implications

fsync latency is governed by journal commit (flush + FUA/barrier to the device); many small fsyncs serialize on it — group commit (batching) is the fix, in ext4 and in every database.

10–11. Interview & Follow-ups

- “Power fails mid-append: what states are possible with/without journaling, per ext4 mode?”
- “Why do databases use O_DIRECT + their own WAL instead of trusting the FS journal?”

12. Coding/Debugging Scenario

fsync-heavy service slow on ext4: check `data=journal` accidentally enabled (double writes), device write-cache/FUA behavior, and batch commits (group multiple records per fsync).

13. Best Practices

App-level durability = your own WAL/fsync discipline; keep FS defaults (ordered) unless measured; put DB WAL on separate low-latency device if fsync-bound.

14. Practice Questions

1. Enumerate post-crash states of “append 4 KB” under: no journal, writeback, ordered, full journal.
2. Design group commit for a WAL that must sustain 50k inserts/s with fsync durability.

7.4 ext4

1. Why Interviewers Ask This

The default Linux FS — asked as “what do you actually run and why”, plus a vehicle for extents/delayed allocation concepts.

2. Core Concept

ext4 = journaling FS with **extents** (contiguous run descriptors instead of per-block pointers), **delayed allocation** (buffer writes, allocate blocks at flush → better contiguity), multiblock allocator, htree directory indexing, and jbd2 journal. Solid general-purpose default; XFS often preferred for huge files/parallelism.

3. Internal Working

- Layout: block groups (superblock/GDT copies, block+inode bitmaps, inode table, data). Flex groups aggregate metadata.
- Extent tree in inode: up to 4 extents inline; deeper trees for fragmented files. An extent maps up to 128 MB contiguous.
- Delayed allocation: dirty pages accumulate; allocation at writeback picks contiguous ranges (this also creates the famous “zero-length file after crash” pattern for apps that skip fsync — mitigated by heuristics for rename patterns).
- Limits: 1 EB volume / 16 TB file (4 KB blocks); timestamps ns-precision to 2446.

4. ASCII Diagram

```
ext2/3 block ptrs: [b1][b2]...[b12] -> [indirect] -> [double] -> [triple]
                  (1000 blocks = 1000+ pointers)
ext4 extents:      [start=8000, len=1000]    (one record!)
Delayed alloc: write() -> dirty pages ... writeback -> allocate 1 big extent
Result: less fragmentation, less metadata, faster fsck.
```

5. Real Production Example

Default on most distro servers and Android (userdata); typical choices: ext4 for general workloads/boot volumes, XFS for large-file/parallel-write servers (RHEL default), ZFS/Btrfs when snapshots/checksums are required.

6. Advantages

Mature/stable, fast recovery (jbd2), extents+delalloc = good contiguity, online resize, wide tooling (e2fsck, tune2fs).

7. Trade-offs

No data checksums (metadata checksums only) — silent bit rot passes; no native snapshots/compression (vs ZFS/Btrfs); fixed inode count at mkfs; delayed allocation vs crash expectations of naive apps.

8. Common Mistakes

- Selling ext4 features it lacks (checksums, snapshots, dedup).
- Not knowing extents vs indirect blocks is *the* ext3 → ext4 headline.
- Ignoring reserved blocks (default 5% for root — “disk full” at 95% for non-root).

9. Performance Implications

Extents shrink metadata I/O for big files dramatically; delalloc turns many small writes into large sequential allocations; jbd2 commit (default 5 s interval or fsync-driven) is the durability heartbeat.

10–11. Interview & Follow-ups

- “ext3 vs ext4?” (extents, delalloc, faster fsck, larger limits) “When XFS/ZFS instead?” “What’s in a block group?”

12. Coding/Debugging Scenario

“Disk full” at 95%: `tune2fs -m` reserved blocks; or inode exhaustion `df -i`; or deleted-open files — the ext4 triage trio.

13. Best Practices

Defaults are good; `noatime/relatime` for read-heavy; align mkfs to workload (inode ratio for small-file storms); LVM under ext4 if you need snapshots.

14. Practice Questions

1. Explain why delayed allocation reduces fragmentation but complicates crash semantics for careless apps.
2. Compare ext4/XFS/ZFS across: checksums, snapshots, max scale, typical use.

7.5 NTFS (High Level)

1. Why Interviewers Ask This

Cross-platform breadth check (Microsoft interviews especially); the interesting content is what design ideas differ from Unix FSes.

2. Core Concept

Windows’ native FS. Central idea: **everything is a file record in the MFT** (Master File Table), and *everything is an attribute* — filename, security descriptor, even data are attributes of an MFT record. Journaling (\$LogFile, metadata-only), B-tree directories, ACL-based security richer than POSIX bits, alternate data streams, compression/encryption built in.

3. Internal Working

- MFT: 1 KB records per file; **small files live entirely inside the record** (“resident data” — very fast small-file access, like inline files).
- Larger data → non-resident attribute with extent-like “data runs”.
- Directories: B-trees of filename attributes (vs classic Unix linear/htree).
- USN change journal (change tracking for indexers/backup), Volume Shadow Copy (snapshot service above FS), hard links + reparse points (symlink/mount equivalents).

4. ASCII Diagram

```
MFT: | rec0 $MFT | rec1 $LogFile | ... | rec N: file.txt |
rec N attributes:
  [$STANDARD_INFO][$FILE_NAME][$SECURITY][$DATA: "hello" (resident)]
big file: [$DATA -> runs: (LCN 8000,len 500),(LCN 12000,len 300)]
Analogy: MFT record ~ inode+dirent+small-data fused; runs ~ extents.
```

5. Real Production Example

Windows servers/desktops, SQL Server hosts; interop via exFAT (removable) since NTFS write support elsewhere is limited; SMB shares expose NTFS ACL semantics that Linux services must map (Samba).

6. Advantages

Rich ACLs, resident small files, change journal, mature online defrag/self-healing, compression/encryption/quotas native.

7. Trade-offs

Proprietary/Windows-centric; MFT fragmentation/zone management; case-insensitive (by default) name semantics — a real bug source when code assumes Unix behavior.

8. Common Mistakes

- Only knowing “it’s the Windows one” — name MFT + attributes + journaling to show breadth.
- Assuming POSIX semantics on NTFS (case-insensitivity, file-locking differences — files open elsewhere can’t be deleted, unlike Unix unlink).

9. Performance Implications

Resident small files beat separate-inode+block designs; ACL checks are richer/heavier; the “can’t delete an open file” model changes deploy patterns (why Windows updates need reboots vs Unix replace-while-running).

10–11. Interview & Follow-ups

- “NTFS vs ext4 — three design differences.” “Why can Linux replace a running binary but Windows can’t?” (unlink semantics vs mandatory sharing locks)

12–14. Scenario / Practices / Questions

- Scenario: a cross-platform build breaks only on Windows: two files differing by case. Fix naming policy.
- Practice: map inode/dentry/journal to their NTFS counterparts (MFT record / \$FILE_NAME index / \$LogFile).

7.6 File Descriptors

1. Why Interviewers Ask This

FDs unify files/sockets/pipes/epoll — “everything is a file descriptor” is the mental model behind all Unix I/O, and FD leaks (“too many open files”) are a top-3 production incident.

2. Core Concept

An FD is a small per-process integer indexing the **process FD table**, whose entries point to entries in the system-wide **open file table** (offset, flags, mode), which point to inodes (or sockets/pipes/etc.). 0/1/2 = stdin/stdout/stderr.

3. Internal Working

- Three levels: FD table (per-process) → open-file description (offset lives HERE) → inode/socket.
- `dup()/dup2()` and `fork()` share one open-file description → **shared offset** (two FDs, one cursor — interview trap!). Two separate `open()`s of the same file → independent offsets.
- `O_CLOEXEC` closes on exec (prevents FD leaks into children).
- Limits: `ulimit -n` per-process (soft/hard), system-wide `fs.file-max`. Sockets, pipes, epoll instances, eventfds, timerfds all consume FDs.

4. ASCII Diagram

```

Process A FD table      Open file table (system)      inodes
fd0 stdin -----+
fd3 -----> | 0FD#7: offset=4096, 0_RDWR |----> inode app.log
fd4 (dup of 3) ---> | (SAME offset!) |
Process B (forked):
fd3 -----> same 0FD#7 -> parent/child share the cursor
Fresh open() by B: 0FD#9: offset=0 -----> same inode
  
```

5. Real Production Example

“Too many open files” (EMFILE) taking down proxies/DB pools: leaked sockets from missing close on error paths; default 1024 soft limit vs 10k connections. Nginx/envoy deployments always raise `nofile`. `lsof -p / ls /proc/<pid>/fd | wc -l` are the triage tools.

6. Advantages

One API (read/write/close/poll) across files, sockets, pipes, devices, timers, events — composability that epoll exploits.

7. Trade-offs

Small-integer namespace leaks easily; shared-offset semantics surprise; limits are per-process and commonly too low by default.

8. Common Mistakes

- Not knowing offset lives in the open-file description (→ can’t explain dup/fork sharing).
- Forgetting sockets/pipes/epoll count against `nofile`.
- Leaking FDs on early-return error paths (use `RAII/defer/try-with-resources`).

9. Performance Implications

FD table lookups are $O(1)$; the real costs are leak-driven exhaustion and, historically, `select()`’s 1024-FD bitmap ceiling (Module 8). Millions of FDs are fine for epoll-based servers with raised limits.

10–11. Interview & Follow-ups

- “Two processes read the same file — do they share the offset? Depends on *how* they got the FDs — explain.”
- “Walk through fixing ‘too many open files’ in production.” “What is `O_CLOEXEC` for?”

12. Coding/Debugging Scenario

Sockets in `CLOSE_WAIT` piling up + FD count climbing → app not closing after peer disconnect; fix close handling; interim: raise limit + alert on `fd_used/fd_limit`.

13. Best Practices

`RAII` everywhere; set `nofile` deliberately per service; monitor FD usage; `O_CLOEXEC` by default (or use `SOCK_CLOEXEC`).

14. Practice Questions

1. Parent opens file, forks; parent reads 100 bytes; where does the child’s next read start? Now both processes `open()` independently — same question.
2. Compute a sane `nofile` for a proxy: 50k conns in, 50k out, plus logs/epoll — show the math and headroom.

7.7 mmap()

1. Why Interviewers Ask This

`mmap` connects Modules 5 and 7 (page cache + demand paging as a file API) and powers real systems (Kafka, RocksDB/LMDB, LLM weight loading) — a perfect senior probe.

2. Core Concept

`mmap(fd)` maps a file’s pages directly into your address space: bytes are accessed with loads/stores instead of read/write syscalls. Pages fault in on demand from the **page cache** — file data with zero copies into user buffers. Modes: `MAP_SHARED` (writes visible to others / write back to file) vs `MAP_PRIVATE` (COW, your own copy); also `MAP_ANONYMOUS` (no file — how malloc gets big blocks).

3. Internal Working

- `mmap` creates a VMA; no I/O yet. First touch → page fault → page cache lookup → (maybe disk read) → PTE maps the *page-cache page itself* into your process. Multiple processes mapping one file share the same physical pages.
- Writes to `MAP_SHARED` dirty page-cache pages; kernel writeback (or `msync`) persists them; the dirty bit tracks it.
- vs `read()`: read copies page cache → your buffer (1 copy + syscall per call); `mmap` = 0 copies, no per-access syscalls, but a page fault per new page and TLB costs.
- `madvise` tunes it: `SEQUENTIAL/WILLNEED` (readahead), `DONTNEED` (drop), `RANDOM`.

4. ASCII Diagram

```
read(): disk -> [page cache] --copy--> user buffer (syscall each time)
mmap(): disk -> [page cache] <--- PTEs map SAME pages into process
        ^ shared by all mappers; writes dirty the page
file bytes 0..N <-> vaddr M..M+N (load/store = file I/O)
Persistence: writeback or msync; MAP_PRIVATE: first write -> COW copy.
```

5. Real Production Example

- **LMDB / SQLite (optional) / RocksDB (optional) / early MongoDB**: B-trees on `mmap` — the OS is the buffer pool.
- **Kafka**: `mmap` for index files.
- LLM inference (llama.cpp): `mmap` weights → instant “load”, pages fault in as used, shared across processes.
- Every process’s code/libraries are `mmaps` of binaries (demand-paged executables).

6. Advantages

Zero-copy reads; page-cache sharing across processes; random access without seek/read choreography; lazy loading; file-backed data structures for free.

7. Trade-offs

Page-fault latency is *invisible* in code (a memory access can block on disk — no place to handle EIO nicely; `SIGBUS` on truncated files!); writeback timing is fuzzy without `msync`; TLB shutdown cost on `munmap`; not great for streaming-once workloads (read+readahead wins); hard-to-account memory (RSS shows shared pages).

8. Common Mistakes

- “`mmap` is always faster than `read`” — per-page faults + TLB churn can lose to batched reads for sequential one-pass I/O.
- Forgetting `SIGBUS` (file truncated by another process while mapped) — must handle for robustness.
- Assuming `MAP_SHARED` writes are durable without `msync`/`fsync`.

9. Performance Implications

Wins: hot random reads on cacheable working sets, many-process sharing, huge sparse files. Loses: cold sequential streaming, tiny files (setup overhead), write-heavy with strict durability (`msync` granularity). DB folklore: “`mmap` considered harmful for DBs” (Andy Pavlo’s paper) — error handling, eviction control, and TLB costs — know both sides.

10–11. Interview & Follow-ups

- “`mmap` vs `read/write` — mechanics and when each wins.” “What happens on the first access to a mapped page?” “`MAP_SHARED` vs `MAP_PRIVATE`?”
- Follow-up: “Why did MongoDB abandon its `mmap` storage engine?” (control over eviction/writeback/durability → `WiredTiger`)

12. Coding/Debugging Scenario

Service crashes with SIGBUS reading a mapped file → another process truncated it; guard with file locks or fstat+bounds, handle SIGBUS, or switch to pread.

13. Best Practices

mmap for shared read-mostly data and file-backed structures; explicit read/write + O_DIRECT for DB-grade control; always plan durability (msync) and SIGBUS handling.

14. Practice Questions

1. Implement a persistent append-only index with mmap: growth (ftruncate+mremap), durability, torn-write handling.
2. Two processes map the same file MAP_SHARED; P1 stores to a page. When does P2 see it? When does the disk? (immediately — same physical page; disk at writeback/msync)

7.8 Buffered vs Direct I/O

1. Why Interviewers Ask This

The capstone of FS interviews: “why do databases use O_DIRECT?” tests page cache, double buffering, durability, and design judgment in one question.

2. Core Concept

- **Buffered I/O** (default): read/write go through the **page cache**; writes return after dirtying RAM; kernel does readahead + writeback batching.
- **Direct I/O** (**O_DIRECT**): DMA straight between your user buffer and the device, bypassing the page cache. Requirements: buffer/offset/length aligned (typically 512 B/4 KB). You own caching, scheduling, and readahead now.
- Orthogonal: **O_SYNC** (buffered but wait for durability per write), **fsync/fdatasync** (flush on demand). O_DIRECT ≠ durable by itself (device cache! still may need flush).

3. Internal Working

Buffered write: copy user → page cache, mark dirty, return; writeback threads flush by age/pressure (**dirty_ratio**, **dirty_expire_centisecs**). Buffered read: page cache hit or miss+readahead. Direct write: pin user pages, DMA to device, return on device ack (not necessarily on media — flush/FUA for that). No copy, no cache pollution, no readahead.

4. ASCII Diagram

```
Buffered:  user buf --copy--> [page cache] --later--> disk
           read: [page cache] --copy--> user buf (+readahead)
Direct:    user buf <===== DMA =====> disk    (no cache, aligned only)
DB double-buffering problem (buffered):
  [InnoDB buffer pool 64GB] + [page cache caching the SAME pages] = waste
=> O_DIRECT for data files; keep WAL buffered-or-O_DSYNC per engine.
```

5. Real Production Example

- MySQL InnoDB: **innodb_flush_method=O_DIRECT** standard practice (avoid double buffering with the buffer pool).
- Postgres: deliberately relies on the page cache (small shared_buffers + OS cache) — the counter-example; know both philosophies.
- Kafka: buffered + sendfile (page cache *is* the design). Video streaming/backup tools: O_DIRECT to avoid trashing the cache for one-pass data.

6. Advantages

Buffered: free caching/readahead/batching, forgiving API. Direct: no double buffering/copy, predictable latency (no writeback storms), cache preserved for others, app-controlled eviction.

7. Trade-offs

Buffered: double-caching with app caches, writeback latency spikes (dirty flush storms), durability requires fsync discipline. Direct: alignment pain, you must rebuild caching/readahead (or your throughput craters), sync writes expose full device latency.

8. Common Mistakes

- “O_DIRECT is faster” unqualified — for most apps buffered wins (readahead + write batching); O_DIRECT wins when *you* cache better than the kernel.
- Believing O_DIRECT implies durability (device write cache — still need flush/FUA or O_DSYNC).
- Forgetting alignment requirements → EINVAL surprises.

9. Performance Implications

Buffered burst writes absorb at RAM speed then stall at `dirty_ratio` (foreground throttling — the mysterious “writes suddenly slow”); O_DIRECT gives flat device-speed latency. Cache-pollution math: one 500 GB backup pass can evict every hot page on the box (`posix_fadvise(DONTNEED)` / O_DIRECT for the backup is the fix).

10–11. Interview & Follow-ups

- “Why does InnoDB use O_DIRECT but Postgres doesn’t?” “What exactly does fsync guarantee vs write?” “How would you stop a backup job from destroying cache hit rates?”

12. Coding/Debugging Scenario

Nightly backup tanks DB p99: page-cache eviction by the backup’s buffered reads → O_DIRECT/fadvise the backup, ionice it, and cap dirty bytes.

13. Best Practices

Default buffered + fsync discipline; O_DIRECT only with your own cache layer; tune `vm.dirty_*` for write-heavy boxes; `fdatasync` over `fsync` when metadata timestamps don’t matter.

14. Practice Questions

1. Design the I/O path for a new LSM storage engine: which files direct, which buffered, where fsync — justify.
2. Explain what each guarantees: `write()` return, `write()+fsync`, O_SYNC write, O_DIRECT write, O_DIRECT+O_DSYNC.

Module 7 Cheat Sheet (one page)

Concept	One-liner	Interview hook
VFS stack	syscalls → VFS → FS → page cache → block → device	where is my write? (RAM until writeback)
Inode	metadata + block map; no name	hard links; deleted-but-open; <code>df -i</code>
Journaling	WAL for FS metadata	ext4 modes: journal/ordered/writeback; ≠ app durability
ext4	extents + delayed alloc + jbd2	vs ext3; no checksums/snapshots
NTFS	MFT records; everything is an attribute	resident small files; can’t delete open files
FDs	<code>fd</code> → open-file-desc(offset!) → inode	dup/fork share offset; EMFILE incidents
mmap	map page cache into address space	zero-copy; SIGBUS; MAP_SHARED vs PRIVATE
Buffered vs Direct	page cache vs DMA bypass	DB double-buffering; fsync semantics

Durability ladder: `write()` = RAM → `+fsync` = on device (+ `dir fsync` for new files) → `O_DIRECT+flush` = bypass cache. **Atomicity:** rename yes, write no. **Triage classics:** `du≠df` → deleted-open files (`ls -lsof +L1`); “No space” with space → `df -i`; EMFILE → `ls /proc/pid/fd | wc -l`.

Top Interview Questions

1. What happens on `open()/read()/write()` end-to-end? When is data durable?
2. Inode contents; hard vs symlink; `rm` of an open file.
3. Power-loss mid-write: outcomes with/without journaling; ext4 modes.
4. Do two processes share a file offset? (`dup/fork` vs independent opens)
5. `mmap` vs `read` — mechanics, when each wins, SIGBUS.
6. Why `O_DIRECT` for InnoDB but page cache for Postgres/Kafka?
7. Debug: “too many open files” / disk-full paradoxes.

Common Mistakes (module-wide)

- `write()` treated as durable; journaling treated as app-level atomicity.
- “Inode stores the filename”; symlink/hardlink confusion.
- Missing shared-offset semantics; forgetting sockets count as FDs.
- “`mmap/O_DIRECT` always faster”; ignoring alignment and SIGBUS.
- Misreading `free -h` (cache ≠ used-up RAM).

Mock Interview (self-test, ~20 min)

1. (Design) Crash-safe key-value store on a raw directory: file layout, WAL, `fsync` points, atomic compaction via `rename`. Defend each choice.
2. (Depth) `echo hi >> file` — enumerate every structure touched (dentry, inode, page cache, journal, bitmaps) and every crash window.
3. (Prod) `df` says 100%, `du` says 60%, and deletes don't help. Full triage script, in order.
4. (Trade-off) Your cache service memory-maps a 200 GB read-mostly dataset. Argue for and against replacing it with `pread` + an app cache.
5. (Trap) “We set `O_DIRECT` so our writes are safe from power loss.” Correct every wrong assumption in that sentence.